The cssbundling-rails tool is a little more complicated. The Tailwind command line tool has two versions, one of which uses Node.js and the other of which is a platform-specific standalone binary. Right now, Rails offers separate tools for each; the cssbundling-rails tool assumes you have Node.js installed, and are using a Node.js-based packaging tool. Because we specified esbuild, Rails assumes we want the Node.js-based tool so it gives us cssbundling-rails. If we were using importmap-rails, it would give us the tailwind-rails gem, which builds CSS using the standalone Tailwind binary. More on this in Chapter 5, *Cascading Style Sheets*.

Package Managers and Modules

Many programming languages have ways for useful libraries of code to be shared along with tools that manage the dependencies between those libraries. The goal is to be able to replicate the exact same versions of libraries in use when installing the application. In Ruby, we have gems, which are managed with Bundler.[27] Bundler works via two files: the Gemfile, which is a manifest of the gems in use, and the Gemfile.lock, which is a list of the actual versions of all gems and their dependencies being loaded by the application.

In JavaScript, the libraries are called *packages*. You'll also sometimes hear them called *modules*, which is technically not correct, even if the directory that Yarn puts all these packages in is called node_modules. There are a few different package managers, of which the first one was the Node Package Manager (npm).[28] We'll use Yarn, which was designed by many of the people who helped build Bundler, and it is somewhat similar in structure. Yarn was created to respond to limitations in npm; however, since then, npm has fixed most of those limitations. Our choice to use Yarn here is largely due to it being the default choice for Rails.

The manifest file for our JavaScript packages is package.json. Let's take a look at the one that Rails has created for us (this is the default file, we'll add a few new packages before we really start coding). (Please note that you can't actually have line breaks in the script lines in the actual .json file.)

```
{
  "name": "app",
```